

UNIVERSITÀ DI NAPOLI “FEDERICO II”



FLUIDODINAMICA NUMERICA

INGEGNERIA AEROSPAZIALE

**Simulazione di flussi viscosi 2D
tramite modello $\psi - \zeta$: Analisi
comparativa di schemi numerici,
ottimizzazioni e tracciamento di
Streaklines**

Author:

Luigi FRAGLIASSO

Supervisor:

Prof. G. COPPOLA



Academic Year 2025-2026

Contents

1	Equazioni di Governo e Descrizione del Modello	4
1.1	Le Equazioni di Navier-Stokes	4
1.2	Ipotesi di Incomprimibilità	5
1.3	Modello $\psi - \zeta$	6
1.3.1	Vorticità (ζ)	6
1.3.2	Funzione di Corrente (ψ)	7
1.3.3	Sistema Adimensionale Finale	7
2	Linee di Flusso	9
2.1	Linee di Corrente (Streamlines)	9
2.2	Traiettorie (Pathlines)	10
2.3	Linee di Emissione (Streaklines)	10
3	Metodologia Numerica e Implementazione	11
3.1	Discretizzazione Spaziale	11
3.1.1	Discretizzazione Del Termine Diffusivo	11
3.1.2	Discretizzazione del Termine Convettivo	11
3.2	Condizioni al Contorno	13
3.3	Gestione della Geometria	15
3.4	Integrazione Temporale	15
3.4.1	Algoritmo di Simulazione e Tracciamento Lagrangiano	16
4	Ottimizzazioni e Varianti Implementative	19
4.1	Risolutore di Poisson: Metodo Iterativo SOR	19
4.2	Accelerazione Hardware: Implementazione C++ (MEX)	20
4.3	Integrazione Temporale Multi-Step (Adams-Bashforth)	20
5	Analisi dei Risultati: Evoluzione della Scia e Conservazione	22
5.1	Fase Iniziale e Transitorio ($t = 0.3 - t = 1.0$)	22
5.2	Regime Completamente Sviluppato ($t = 5.0s$)	23
5.3	Analisi dell'Enstrofia e Singolarità Iniziale	24
6	Implementazione e Analisi dei Casi Studio	25
6.1	Setup della Simulazione	25
6.2	Analisi dei Solutori per l'Equazione di Poisson	26
6.2.1	Efficienza: Tempo Medio di Calcolo per Step ($\bar{T}_{step}$)	26
6.3	Analisi Comparativa delle Strategie di Integrazione Temporale	27

6.3.1	Confronto di Stabilità e Robustezza	28
6.4	Confronto Complessivo e Conclusioni	30
7	Presentation of the Code	31

Abstract

Il presente elaborato discute la simulazione numerica di un flusso viscoso incompressibile bidimensionale attorno ad un ostacolo. Le equazioni governanti di Navier-Stokes sono risolte utilizzando la formulazione del modello $\psi - \zeta$ su una griglia collocata e discretizzata spazialmente tramite Differenze Finite. Per quanto concerne la risoluzione numerica, vengono esplorati e confrontati diversi approcci implementativi: l'integrazione temporale è affrontata sia con metodi *single-step* (Runge-Kutta) che *multi-step* (Adams-Bashforth), mentre per la risoluzione dell'equazione ellittica si analizza l'efficienza di solutori iterativi (SOR), anche mediante ottimizzazioni in linguaggio C++. L'obiettivo principale del lavoro è l'implementazione di una procedura lagrangiana per la visualizzazione della scia tramite il calcolo delle *streaklines* (linee di emissione). Queste vengono generate rilasciando particelle traccianti in istanti di tempo successivi da uno stesso punto di iniezione $\mathbf{x}_0$; congiungendo le posizioni attuali (dette *posizioni isocrone*) di tutte le particelle precedentemente rilasciate, si ottiene la rappresentazione istantanea della linea di emissione.

1 Equazioni di Governo e Descrizione del Modello

In questo capitolo vengono descritte le equazioni fondamentali che governano il moto dei fluidi. Si parte dalla formulazione più generale delle equazioni di Navier-Stokes (conservazione di massa, quantità di moto ed energia) per poi introdurre le ipotesi semplificative di incomprimibilità che portano al disaccoppiamento dell'equazione dell'energia. Infine, il sistema viene particolarizzato per il caso bidimensionale (2D) utilizzando il modello $\psi - \zeta$.

1.1 Le Equazioni di Navier-Stokes

Le equazioni fondamentali che descrivono il moto di un fluido viscoso newtoniano sotto l'influenza di pressione, forze viscose e forze esterne sono le **equazioni di Navier-Stokes**. In un sistema di riferimento Euleriano, indicando con $\mathbf{V}(\mathbf{x}, t)$ il vettore velocità, le equazioni di conservazione in forma vettoriale sono:

Conservazione della Massa (Continuità):

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0 \quad (1)$$

Conservazione della Quantità di Moto:

$$\frac{\partial(\rho \mathbf{V})}{\partial t} + \nabla \cdot (\rho \mathbf{V} \mathbf{V}) = \nabla \cdot \boldsymbol{\tau} + \rho \mathbf{f} \quad (2)$$

dove:

- ρ è la densità del fluido,
- $\mathbf{f}$ rappresenta le forze di volume esterne,
- $\boldsymbol{\tau}$ è il tensore degli sforzi.

Il tensore degli sforzi $\boldsymbol{\tau}$ per un fluido newtoniano si decompone in una parte isotropa (pressione) e una parte deviatorica (viscosa):

$$\boldsymbol{\tau} = -p\mathbf{I} + \boldsymbol{\tau}_d \quad (3)$$

dove $\boldsymbol{\tau}_d$, assumendo l'ipotesi di Stokes, è legato al tasso di deformazione dalla viscosità dinamica μ :

$$\boldsymbol{\tau}_d = 2\mu \left[\frac{1}{2} (\nabla \mathbf{V} + (\nabla \mathbf{V})^T) - \frac{1}{3} (\nabla \cdot \mathbf{V}) \mathbf{I} \right] \quad (4)$$

Conservazione dell'Energia Totale:

$$\frac{\partial(\rho E)}{\partial t} + \nabla \cdot (\rho E \mathbf{u}) = \nabla \cdot (\boldsymbol{\tau} \cdot \mathbf{u}) - \nabla \cdot \mathbf{J}_q + \rho \mathbf{f} \cdot \mathbf{u} \quad (5)$$

dove:

- $E = e + \frac{1}{2} |\mathbf{V}|^2$ è l'energia totale per unità di volume data dalla somma di energia interna e ed energia cinetica).
- $\mathbf{J}_q = -k \nabla T$ è il flusso termico descritto dalla Legge di Fourier, con k conducibilità termica e T temperatura.

1.2 Ipotesi di Incomprimibilità

L'ipotesi di fluido **incompressibile** implica che la densità ρ sia costante nel tempo e nello spazio. Di conseguenza, l'equazione di continuità si riduce a un vincolo cinematico sul campo di velocità, che deve essere a divergenza nulla ($\nabla \cdot \mathbf{V} = 0$). L'adozione di questa ipotesi comporta una fondamentale modifica nella natura delle equazioni: l'equazione di stato termodinamica (che lega pressione, densità e temperatura) viene meno. Se così non fosse, la pressione sarebbe determinata univocamente dalla temperatura e non vi sarebbero gradi di libertà sufficienti per soddisfare il vincolo di incomprimibilità. Nel modello incompressibile, la pressione perde il suo significato termodinamico e diventa una variabile puramente meccanica (un moltiplicatore di Lagrange) che si adatta istantaneamente per garantire che il campo di moto rimanga solenoidale. Questo porta al **disaccoppiamento** dell'equazione dell'energia dal sistema dinamico. La temperatura T diviene uno scalare passivo e il sistema da risolvere si riduce alle sole equazioni di massa e quantità di moto:

$$\begin{cases} \nabla \cdot \mathbf{V} = 0 \\ \frac{\partial \mathbf{V}}{\partial t} + (\mathbf{V} \cdot \nabla) \mathbf{V} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{V} + \mathbf{f} \end{cases} \quad (6)$$

1.3 Modello $\psi - \zeta$

La risoluzione del sistema (6) nelle variabili primitive $(\mathbf{V}, p)$ presenta difficoltà numeriche, legate principalmente all'assenza di un'equazione evolutiva esplicita per la pressione. Tuttavia, limitandosi allo studio di flussi **bidimensionali**, è possibile riformulare il problema in modo più efficiente introducendo due nuove variabili scalari: la **Vorticità** (ζ) e la **Funzione di Corrente** (ψ).

L'introduzione del modello $\psi - \zeta$ è motivata da decisive semplificazioni analitiche e numeriche:

1. **Eliminazione della pressione:** Attraverso l'uso della vorticità, la pressione scompare dalle equazioni governanti, rimuovendo la necessità di risolvere l'accoppiamento velocità-pressione.
2. **Soddisfacimento automatico della continuità:** La definizione della funzione di corrente garantisce identicamente che la condizione $\nabla \cdot \mathbf{V} = 0$ sia rispettata, senza dover risolvere un'equazione differenziale dedicata.
3. **Riduzione delle incognite:** Il problema vettoriale viene trasformato in un sistema di due equazioni scalari accoppiate.

1.3.1 Vorticità (ζ)

La vorticità è definita vettorialmente come il rotore del campo di velocità:

$$\boldsymbol{\omega} = \nabla \times \mathbf{u} \quad (7)$$

Questa definizione permette di riscrivere il termine convettivo dell'equazione della quantità di moto utilizzando l'identità vettoriale:

$$\mathbf{u} \cdot \nabla \mathbf{u} = \boldsymbol{\omega} \times \mathbf{u} + \nabla(|\mathbf{u}|^2/2). \quad (8)$$

Applicando l'operatore rotore ($\nabla \times$) a tutta l'equazione della quantità di moto, il termine di gradiente di pressione scompare (poiché $\nabla \times \nabla p = 0$). Si ottiene così l'equazione di trasporto della vorticità:

$$\frac{\partial \boldsymbol{\omega}}{\partial t} + \mathbf{u} \cdot \nabla \boldsymbol{\omega} - \boldsymbol{\omega} \cdot \nabla \mathbf{u} = \nu \nabla^2 \boldsymbol{\omega} \quad (9)$$

Nel caso specifico di un flusso **bidimensionale** (sul piano xy), il vettore vorticità è sempre perpendicolare al piano del moto. Definendo la componente scalare ω_z , l'equazione si semplifica notevolmente in quanto il termine di **stretching** ($\boldsymbol{\omega} \cdot \nabla \mathbf{u}$) si annulla. Definendo la variabile:

$$\zeta = -\omega_z \quad (10)$$

opposta della vorticità fisica per convenzione del modello, l'equazione scalare di trasporto diventa:

$$\frac{\partial \zeta}{\partial t} + \mathbf{u} \cdot \nabla \zeta = \nu \nabla^2 \zeta \quad (11)$$

1.3.2 Funzione di Corrente (ψ)

Poiché il flusso è incompressibile, il campo di velocità è a divergenza nulla ($\nabla \cdot \mathbf{u} = 0$). Questo implica l'esistenza di un potenziale vettore $\boldsymbol{\psi}$ tale che $\mathbf{u} = \nabla \times \boldsymbol{\psi}$. In due dimensioni, questo potenziale ha solo la componente normale al piano, detta **funzione di corrente** scalare ψ . Le componenti della velocità sono legate a ψ dalle relazioni:

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x} \quad (12)$$

Questa definizione soddisfa identicamente l'equazione di continuità. Sostituendo le relazioni (12) nella definizione di vorticità ($\omega_z = \partial v / \partial x - \partial u / \partial y$), si ottiene una relazione ellittica di Poisson che lega la cinematica del campo:

$$\nabla^2 \psi = -\omega_z = \zeta \quad (13)$$

1.3.3 Sistema Adimensionale Finale

Al fine di generalizzare i risultati della simulazione e renderli indipendenti dalla scala fisica specifica del problema, è necessario operare una adimensionalizzazione delle equazioni governanti. Si introducono le grandezze adimensionali (indicate con l'asterisco) normalizzando le variabili fisiche rispetto a una lunghezza caratteristica L_{ref} e una velocità di riferimento u_{ref} :

$$\mathbf{x}^* = \frac{\mathbf{x}}{L_{ref}}, \quad \mathbf{u}^* = \frac{\mathbf{u}}{u_{ref}}, \quad t^* = \frac{t u_{ref}}{L_{ref}}, \quad \zeta^* = \frac{\zeta L_{ref}}{u_{ref}}, \quad \psi^* = \frac{\psi}{L_{ref} u_{ref}}$$

Sostituendo queste definizioni nell'equazione di trasporto, emerge l'unico parametro fisico adimensionale che governa la dinamica del flusso viscoso, il **Numero di Reynolds**:

$$Re = \frac{u_{ref} L_{ref}}{\nu} \quad (14)$$

Il sistema finale $\psi - \zeta$ in forma adimensionale (omettendo d'ora in poi gli asterischi per semplicità di notazione) che verrà risolto numericamente è costituito dalle seguenti due equazioni accoppiate:

$$\begin{cases} \frac{\partial \zeta}{\partial t} + \mathbf{u} \cdot \nabla \zeta = \frac{1}{Re} \nabla^2 \zeta & \text{(Eq. di Trasporto)} \\ \nabla^2 \psi = \zeta & \text{(Eq. di Poisson)} \end{cases} \quad (15)$$

2 Linee di Flusso

Oltre alla determinazione dei campi scalari e vettoriali, un aspetto cruciale della fluidodinamica computazionale è la visualizzazione del moto. Per comprendere la topologia del flusso, specialmente in regimi instazionari (dipendenti dal tempo), è fondamentale distinguere tra diverse curve caratteristiche: linee di corrente, traiettorie e linee di emissione. Sebbene in un flusso stazionario queste tre curve coincidano, nel caso di flussi instazionari esse differiscono sostanzialmente e forniscono informazioni fisiche diverse.

2.1 Linee di Corrente (Streamlines)

Una linea di corrente è definita come una curva istantanea tangente in ogni punto al vettore velocità $\mathbf{u}(\mathbf{x}, t)$ in un fissato istante t . Essa fornisce un'immagine "fotografica" del campo di moto euleriano congelato nel tempo. Matematicamente, indicando con $d\mathbf{l}$ l'elemento infinitesimo orientato lungo la linea, la condizione di tangenza si esprime come prodotto vettoriale nullo:

$$\mathbf{u} \times d\mathbf{l} = 0 \quad (16)$$

Nel contesto del modello $\psi - \zeta$ bidimensionale, le linee di corrente coincidono con le curve di livello della funzione scalare ψ . Per dimostrarlo, consideriamo il gradiente di ψ , che è un vettore giacente nel piano del moto $x - y$:

$$\nabla\psi = \left(\frac{\partial\psi}{\partial x}, \frac{\partial\psi}{\partial y} \right) \quad (17)$$

Effettuando il prodotto scalare tra il vettore velocità $\mathbf{u} = (u, v)$ e il gradiente di ψ , e sostituendo le definizioni $u = \partial\psi/\partial y$ e $v = -\partial\psi/\partial x$, si ottiene:

$$\mathbf{u} \cdot \nabla\psi = u \frac{\partial\psi}{\partial x} + v \frac{\partial\psi}{\partial y} = \left(\frac{\partial\psi}{\partial y} \right) \frac{\partial\psi}{\partial x} + \left(-\frac{\partial\psi}{\partial x} \right) \frac{\partial\psi}{\partial y} = 0 \quad (18)$$

Il risultato nullo del prodotto scalare implica che il vettore velocità è ovunque **ortogonale** al gradiente della funzione di corrente ($\mathbf{u} \perp \nabla\psi$). Poiché, per definizione, la velocità $\mathbf{u}$ è tangente alle linee di corrente, ne consegue che il gradiente $\nabla\psi$ (che rappresenta la direzione di massima variazione dello scalare) è normale alle linee di corrente. Ciò significa che la variazione di ψ lungo la direzione del moto è nulla. Pertanto, ψ si mantiene costante lungo ogni linea di corrente ($\psi = \text{cost}$), le quali coincidono geometricamente con le curve di livello (isolinee) della funzione di corrente.

2.2 Traiettorie (Pathlines)

Una traiettoria è il luogo geometrico dei punti visitati da una singola particella fluida nel corso del suo moto. È un concetto intrinsecamente lagrangiano che descrive la storia temporale di un elemento fluido. Indicando con $\mathbf{x}_p(t)$ la posizione della particella, la traiettoria è determinata dall'integrazione nel tempo della velocità locale:

$$\frac{d\mathbf{x}_p}{dt} = \mathbf{u}(\mathbf{x}_p(t), t), \quad \text{con } \mathbf{x}_p(t_0) = \mathbf{x}_{start} \quad (19)$$

A differenza delle linee di corrente, le traiettorie non dipendono solo dal campo istantaneo, ma dall'intera evoluzione temporale del campo di velocità.

2.3 Linee di Emissione (Streaklines)

Le linee di emissione, o *streaklines*, sono il luogo geometrico delle posizioni occupate in un dato istante t da tutte le particelle fluide che sono transitate per un determinato punto spaziale fisso $\mathbf{x}_{in}$ (punto di iniezione) in istanti passati diversi. Questa definizione corrisponde esattamente a ciò che si osserva sperimentalmente iniettando in modo continuo un colorante (o fumo) in un punto del fluido: la linea visibile è formata dall'insieme delle particelle di colorante rilasciate in precedenza. Dal punto di vista matematico e computazionale, una streakline all'istante t è costruita unendo le **posizioni isocrone** $\mathbf{x}_k(t)$ di una serie di particelle k , ciascuna rilasciata al tempo $t_k = t_0 + k\Delta t$. Ogni particella soddisfa la propria equazione di traiettoria:

$$\begin{cases} \frac{d\mathbf{x}_k}{dt} = \mathbf{u}(\mathbf{x}_k(t), t) \\ \mathbf{x}_k(t_k) = \mathbf{x}_{in} \end{cases} \quad (20)$$

La visualizzazione delle streaklines è l'obiettivo primario di questo lavoro, in quanto permette di evidenziare strutture coerenti e la dinamica della scia (come i vortici di von Kármán) in modo più efficace rispetto alle semplici linee di corrente istantanee.

3 Metodologia Numerica e Implementazione

La soluzione del sistema governante (15) non è ottenibile analiticamente per geometrie arbitrarie; si ricorre pertanto all'analisi numerica. In questa sezione vengono descritte le tecniche di discretizzazione spaziale e temporale adottate per trasformare le equazioni alle derivate parziali in un sistema algebrico risolvibile al calcolatore.

3.1 Discretizzazione Spaziale

Il dominio fisico continuo è discretizzato mediante una griglia cartesiana strutturata e uniforme (griglia *collocated*), con passo spaziale costante $h = \Delta x = \Delta y$. Le variabili ψ e ζ sono definite nei nodi geometrici della griglia (i, j) . Le derivate spaziali sono approssimate utilizzando schemi alle **Differenze Finite** (Finite Difference Method, FDM) centrati del secondo ordine.

3.1.1 Discretizzazione Del Termine Diffusivo

Per l'operatore di Laplace (∇^2), si utilizza la classica "stella a 5 punti":

$$\nabla^2 f|_{i,j} \approx \frac{f_{i+1,j} + f_{i-1,j} + f_{i,j+1} + f_{i,j-1} - 4f_{i,j}}{h^2} \quad (21)$$

3.1.2 Discretizzazione del Termine Convettivo

La discretizzazione del termine non lineare convettivo è un aspetto critico per la stabilità e la fedeltà fisica della simulazione. Analiticamente, tale termine può essere espresso in due forme equivalenti (grazie all'incomprimibilità $\nabla \cdot \mathbf{u} = 0$):

- **Forma Advective:** $\mathbf{u} \cdot \nabla \zeta$
- **Forma Divergence:** $\nabla \cdot (\mathbf{u} \zeta)$

L'utilizzo dell'una o dell'altra forma in ambito discreto dipende dalle proprietà di conservazione che lo schema numerico deve rispettare. Utilizzando uno schema alle differenze finite centrato su griglia uniforme, le discretizzazioni assumono la seguente forma:

1. Forma Advective:

$$\mathbf{u} \cdot \nabla \zeta|_{i,j} \approx u_{i,j} \frac{\zeta_{i+1,j} - \zeta_{i-1,j}}{2h} + v_{i,j} \frac{\zeta_{i,j+1} - \zeta_{i,j-1}}{2h} \quad (22)$$

2. Forma Divergence:

$$\nabla \cdot (\mathbf{u}\zeta)|_{i,j} \approx \frac{u_{i+1,j}\zeta_{i+1,j} - u_{i-1,j}\zeta_{i-1,j}}{2h} + \frac{v_{i,j+1}\zeta_{i,j+1} - v_{i,j-1}\zeta_{i,j-1}}{2h} \quad (23)$$

L'obiettivo è che la discretizzazione non introduca grandezze spurie nel dominio; si richiede cioè la conservazione degli invarianti fisici: l'**invariante lineare** (la vorticità totale ζ) e l'**invariante quadratico** (l'enstrofia $\mathcal{E} \propto \zeta^2$). Affinché ciò avvenga, l'integrale globale sul dominio Ω del termine convettivo deve annullarsi (assumendo flussi al bordo nulli o periodici):

$$\int_{\Omega} \nabla \cdot (\mathbf{u}\zeta) d\Omega = 0 \quad \text{e} \quad \int_{\Omega} \mathbf{u} \cdot \nabla \zeta d\Omega = 0 \quad (24)$$

Analisi dell'Invariante Lineare: Per valutare la conservazione delle proprietà fisiche, è utile esprimere i termini convettivi discreti in notazione matriciale. Introduciamo le matrici diagonali delle velocità $U = \text{diag}(u_{i,j})$ e $V = \text{diag}(v_{i,j})$, e le matrici D_x e D_y rappresentanti gli operatori di derivazione discreta (differenze centrali). Le due forme del termine convettivo si scrivono quindi come operazioni matriciali sul vettore vorticità ζ :

- **Forma Divergence:** corrisponde al vettore $(D_x U + D_y V)\zeta$.
- **Forma Advective:** corrisponde al vettore $(U D_x + V D_y)\zeta$.

Si dimostra che la *Forma Divergence* conserva l'invariante lineare intrinsecamente (la somma telescopica dei flussi si annulla), senza necessità di ulteriori ipotesi. La *Forma Advective*, invece, lo conserva solo sotto l'ipotesi di utilizzo di *schemi centrali* e se il campo di velocità discreto soddisfa esattamente la condizione di *incompressibilità*.

Analisi dell'Invariante Quadratico (Enstrofia): La conservazione dell'enstrofia richiede che il termine convettivo non contribuisca alla variazione dell'energia rotazionale del sistema. In termini algebrici, ciò implica che le forme quadratiche associate siano nulle. Tuttavia, utilizzando le notazioni introdotte, si osserva che:

- Per la forma Divergence, la forma quadratica associata è $\zeta^T(D_x U + D_y V)\zeta$. Questa non è nulla poiché la matrice risultante non è antisimmetrica.
- Per la forma Advective, la forma quadratica associata è $\zeta^T(U D_x + V D_y)\zeta$. Analogamente, anche questa non si annulla in generale.

Nessuna delle due forme fondamentali, presa singolarmente, è in grado di conservare l'energia, portando a potenziali instabilità numeriche non lineari per tempi lunghi.

Soluzione: Forma Skew-Symmetric Si dimostra che i due schemi presentano un errore di discretizzazione rispetto alla conservazione dell'energia che è uguale in modulo ma di segno opposto. Pertanto, operando una combinazione lineare delle due forme con coefficiente $\alpha = 0.5$:

$$\text{Conv}_{Skew} = \frac{1}{2}(\text{Advective}) + \frac{1}{2}(\text{Divergence}) \quad (25)$$

si ottiene la forma **Skew-Symmetric**. Questa formulazione garantisce che la matrice discreta risultante sia antisimmetrica, assicurando identicamente la conservazione dell'invariante quadratico (energia) e garantendo la stabilità robusta della simulazione.

3.2 Condizioni al Contorno

Il problema ellittico-parabolico richiede condizioni specifiche sulle frontiere del dominio computazionale.

Funzione di Corrente (ψ): Sulle pareti solide (sia i muri del canale che la superficie dell'ostacolo), la velocità normale deve essere nulla. Poiché la differenza di valore di ψ tra due punti rappresenta la portata che passa tra essi, le pareti solide sono linee di corrente a valore costante ($\psi = \text{cost}$).

- **Pareti canale (Sud/Nord):** Si impone $\psi_{sud} = 0$ come valore di riferimento e $\psi_{nord} = Q$. Sebbene la funzione di corrente sia definita a meno di una costante arbitraria, la differenza di valore tra due linee di corrente, come detto in precedenza, rappresenta la portata volumetrica che scorre tra di esse. Di conseguenza, è necessario imporre un salto pari a Q tra le pareti; se avessimo imposto lo stesso valore su entrambi

i lati (ad esempio entrambi nulli), la portata netta risultante sarebbe stata zero, impedendo lo sviluppo di un flusso che scorre da sinistra verso destra.

- **Ostacolo:** Sulla superficie dell'ostacolo si impone $\psi_{ost} = Q/2$. Questo valore è scelto per ragioni di simmetria e continuità del flusso. Se avessimo imposto $\psi_{ost} = 0$ (uguale alla parete Sud), l'intera regione compresa tra la parete inferiore e l'ostacolo sarebbe diventata un'unica linea di corrente a valore costante; ciò avrebbe impedito fisicamente il passaggio del fluido in quella zona, forzando l'intero flusso a scorrere esclusivamente nella parte superiore del canale, alterando drasticamente la fisica del problema.
- **Ingresso/Uscita (Est/Ovest):** Si impone un profilo lineare $\psi(y) = u_{ref}y$. Questa scelta genera un campo di velocità uniforme in ingresso; infatti, ricordando la definizione $u = \partial\psi/\partial y$, la pendenza costante del profilo lineare corrisponde esattamente alla velocità media, calcolata come rapporto tra la differenza di portata ai bordi e l'altezza del dominio:

$$u_{ref} = \frac{\partial\psi}{\partial y} = \frac{\psi_{nord} - \psi_{sud}}{L_y} = \frac{Q - 0}{L_y} = \frac{Q}{L_y}$$

Vorticità (ζ): Per la vorticità non esiste una condizione fisica diretta al contorno. Si utilizza pertanto una relazione numerica derivata dall'espansione in serie di Taylor della funzione di corrente in prossimità della parete esterna e dell'ostacolo. Considerando un nodo fluido n adiacente a un nodo di parete w , lo sviluppo al secondo ordine è:

$$\psi_n = \psi_w + \left. \frac{\partial\psi}{\partial n} \right|_w h + \left. \frac{\partial^2\psi}{\partial n^2} \right|_w \frac{h^2}{2} + \mathcal{O}(h^3) \quad (26)$$

Ricordando che la velocità tangenziale è $u_w = (\partial\psi/\partial n)_w$ e che, per l'equazione di Poisson alla parete, $\zeta_w = (\partial^2\psi/\partial n^2)_w$, si può esplicitare la vorticità a parete:

$$\zeta_w = \frac{2(\psi_n - \psi_w - u_w h)}{h^2} + \mathcal{O}(h) \quad (27)$$

Imponendo la condizione fisica di aderenza (**no-slip**) sulle pareti solide, la velocità tangenziale si annulla ($u_w = 0$). Si ottiene così la celebre **Formula**

di Thom:

$$\zeta_w = \frac{2(\psi_n - \psi_w)}{h^2} \quad (28)$$

Questa formula permette alla vorticità di essere generata fisicamente sulla superficie del corpo (a causa dei gradienti di velocità) e successivamente diffusa e convettata nella scia.

3.3 Gestione della Geometria

La presenza di ostacoli immersi nel flusso (cilindro o profilo alare) all'interno di una griglia cartesiana rettangolare è gestita tramite una matrice topologica, denominata $\mathbf{G}$. Questa matrice funge da "maschera" per distinguere la natura di ciascun nodo (i, j) :

- **Dominio Fluidico** ($G > 0$): Nodi in cui le equazioni di trasporto e di Poisson vengono risolte attivamente.
- **Ostacolo** ($G = -1$): Nodi interni al corpo solido, esclusi dal calcolo fluido.
- **Bordi** ($G = 0$): Nodi di frontiera dove vengono imposte le condizioni al contorno.

Questa tecnica permette di trattare geometrie anche più complesse semplicemente modificando la definizione iniziale della matrice $\mathbf{G}$, senza alterare il solutore delle equazioni.

3.4 Integrazione Temporale

La discretizzazione spaziale conduce a un sistema di Equazioni Differenziali Ordinarie (ODEs) per i valori nodali della vorticità ζ . Definendo l'operatore spaziale $\mathcal{F}(\zeta)$ che ingloba il termine convettivo (matrice $\mathbf{C}$) e il termine diffusivo (matrice Laplaciana $\mathbf{L}$):

$$\mathcal{F}(\zeta) = -\mathbf{C}(\zeta)\zeta + \frac{1}{Re}\mathbf{L}\zeta \quad (29)$$

l'evoluzione temporale è governata dall'equazione $d\zeta/dt = \mathcal{F}(\zeta)$. Per l'avanzamento temporale si adotta il metodo esplicito **Runge-Kutta al quarto ordine (RK4)**. Questo schema è definito dal seguente *Tableau di Butcher*, che specifica i pesi e i nodi temporali per i quattro stadi intermedi:

$$\begin{array}{c|cccc}
0 & 0 & 0 & 0 & 0 \\
1/2 & 1/2 & 0 & 0 & 0 \\
1/2 & 0 & 1/2 & 0 & 0 \\
1 & 0 & 0 & 1 & 0 \\
\hline
& 1/6 & 1/3 & 1/3 & 1/6
\end{array} \tag{30}$$

Dato lo stato noto al tempo t_n , indicato con ζ_{old} , l'algoritmo calcola la soluzione al passo successivo ζ_{new} attraverso la valutazione di quattro pendenze intermedie (f_1, f_2, f_3, f_4) e stati intermedi predetti ($\zeta_1, \zeta_2, \zeta_3, \zeta_4$). La procedura esplicita per un passo Δt è la seguente:

$$\begin{aligned}
\text{Stadio 1: } \quad \zeta_1 &= \zeta_{old} \\
f_1 &= \mathcal{F}(\zeta_1)
\end{aligned} \tag{31}$$

$$\begin{aligned}
\text{Stadio 2: } \quad \zeta_2 &= \zeta_{old} + \frac{\Delta t}{2} f_1 \\
f_2 &= \mathcal{F}(\zeta_2)
\end{aligned} \tag{32}$$

$$\begin{aligned}
\text{Stadio 3: } \quad \zeta_3 &= \zeta_{old} + \frac{\Delta t}{2} f_2 \\
f_3 &= \mathcal{F}(\zeta_3)
\end{aligned} \tag{33}$$

$$\begin{aligned}
\text{Stadio 4: } \quad \zeta_4 &= \zeta_{old} + \Delta t f_3 \\
f_4 &= \mathcal{F}(\zeta_4)
\end{aligned} \tag{34}$$

La soluzione aggiornata al tempo t_{n+1} è infine ottenuta mediante una media pesata delle pendenze calcolate:

$$\zeta_{new} = \zeta_{old} + \Delta t \left[\frac{1}{6}(f_1 + f_4) + \frac{1}{3}(f_2 + f_3) \right] \tag{35}$$

3.4.1 Algoritmo di Simulazione e Tracciamento Lagrangiano

La medesima strategia di integrazione RK4 viene applicata per evolvere la posizione delle particelle traccianti $\mathbf{x}_p$ per la visualizzazione delle streaklines. L'algoritmo di simulazione procede iterativamente secondo i seguenti passi:

1. **Aggiornamento Vorticità al Bordo:** Prima di ogni stadio del RK4, i valori di ζ sui nodi di parete vengono ricalcolati utilizzando la formula di Thom.
2. **Integrazione Euleriana:** Si calcola il nuovo campo di vorticità ζ_{new} risolvendo l'equazione di trasporto secondo lo schema RK4 sopra descritto.
3. **Risoluzione Ellittica:** Si aggiorna il campo cinematico risolvendo il sistema lineare sparso associato all'equazione di Poisson:

$$\mathbf{L}\psi_{new} = \zeta_{new} \implies \psi_{new} = \mathbf{L}^{-1}\zeta_{new} \quad (36)$$

4. **Calcolo Velocità:** Si deriva il campo di velocità discreto $\mathbf{U}_{grid}$ sui nodi (i, j) applicando le differenze centrali alla funzione di corrente aggiornata.
5. **Integrazione Lagrangiana (Streaklines):** Parallelamente, si traccia il moto delle particelle passive risolvendo $d\mathbf{x}_p/dt = \mathbf{u}(\mathbf{x}_p)$. Poiché la posizione della particella $\mathbf{x}_p$ varia nel continuo e non coincide quasi mai con i nodi della griglia, la velocità $\mathbf{u}(\mathbf{x}_p)$ necessaria per calcolare le pendenze f_i del RK4 deve essere ricostruita localmente.

Si utilizza un'interpolazione bilineare basata sui quattro nodi della cella (i, j) che ospita la particella. L'idea alla base di questo metodo è approssimare l'andamento della velocità come lineare lungo le due direzioni assiali all'interno di ogni cella, garantendo la continuità del campo (C^0) attraverso i bordi degli elementi.

Definite le coordinate locali normalizzate $\tilde{x} = (x_p - x_i)/h$ e $\tilde{y} = (y_p - y_j)/h$, che variano tra 0 e 1, la velocità interpolata è espressa in forma matriciale come:

$$\mathbf{u}(\mathbf{x}_p) \approx \begin{bmatrix} 1 - \tilde{x} & \tilde{x} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{u}_{i,j} & \mathbf{u}_{i,j+1} \\ \mathbf{u}_{i+1,j} & \mathbf{u}_{i+1,j+1} \end{bmatrix} \cdot \begin{bmatrix} 1 - \tilde{y} \\ \tilde{y} \end{bmatrix} \quad (37)$$

Interpretazione fisica dei pesi: La formula rappresenta una media pesata delle velocità ai vertici. I coefficienti agiscono come pesi geometrici inversamente proporzionali alla distanza tra la particella e il nodo:

- Il termine $(1 - \tilde{x})(1 - \tilde{y})$ è il peso associato al nodo (i, j) . Se la particella si trova esattamente sul nodo (i, j) (quindi $\tilde{x} = 0, \tilde{y} = 0$), questo peso vale 1 e la velocità interpolata coincide esattamente con quella nodale.
- Man mano che la particella si allontana da un nodo, il peso corrispondente diminuisce linearmente, trasferendo "influenza" ai nodi verso cui ci si sta avvicinando.

Questa procedura assicura che la particella risenta maggiormente del campo di velocità del nodo a essa più prossimo, permettendo una ricostruzione fluida delle traiettorie anche su griglie discrete.

Questo valore di velocità istantanea alimenta gli stadi del Runge-Kutta per l'aggiornamento della posizione $\mathbf{x}_p$.

4 Ottimizzazioni e Varianti Implementative

Oltre all'implementazione standard descritta nei capitoli precedenti, sono state esplorate e integrate nel codice alcune varianti numeriche e ottimizzazioni software. Queste modifiche mirano a migliorare l'efficienza computazionale (riduzione dei tempi di calcolo e dell'occupazione di memoria) o a sperimentare diverse strategie di integrazione.

4.1 Risolutore di Poisson: Metodo Iterativo SOR

Nell'implementazione base, l'equazione di Poisson per la funzione di corrente ($\nabla^2\psi = \zeta$) viene risolta mediante un metodo diretto, che prevede l'inversione (o la fattorizzazione) della grande matrice sparsa Laplaciana $\mathbf{L}$. Sebbene molto efficiente per griglie di dimensioni moderate, questo approccio presenta limiti di scalabilità:

- **Occupazione di Memoria:** La matrice $\mathbf{L}$, pur essendo sparsa, richiede l'allocazione di una notevole quantità di RAM quando il numero di nodi $N = N_x \times N_y$ cresce (complessità spaziale non trascurabile per l'indicizzazione).
- **Tempo di Calcolo:** La fattorizzazione diretta diviene computazionalmente onerosa all'aumentare della finezza della griglia.

Per ovviare a tali problematiche su mesh molto fitti, è stato implementato un risolutore alternativo basato sul metodo iterativo **SOR (Successive Over-Relaxation)**. L'algoritmo aggiorna il valore di $\psi_{i,j}$ localmente senza mai assemblare la matrice globale:

$$\psi_{i,j}^{k+1} = (1 - \omega)\psi_{i,j}^k + \omega \frac{\psi_{i+1,j}^k + \psi_{i-1,j}^{k+1} + \psi_{i,j+1}^k + \psi_{i,j-1}^{k+1} - h^2\zeta_{i,j}}{4} \quad (38)$$

dove ω è il fattore di rilassamento ($1 < \omega < 2$) scelto per accelerare la convergenza. Sebbene per le griglie utilizzate in questo studio i vantaggi prestazionali siano marginali, l'approccio iterativo garantisce un'occupazione di memoria $O(N)$ (lineare), risultando vincente per simulazioni ad alta risoluzione (High-Performance Computing).

4.2 Accelerazione Hardware: Implementazione C++ (MEX)

I metodi iterativi come il SOR richiedono di scorrere tutti i nodi della griglia per migliaia di iterazioni ad ogni passo temporale. In un linguaggio interpretato come MATLAB, l'esecuzione di cicli `for` annidati comporta un notevole **overhead** (costo di interpretazione) che rallenta drasticamente la simulazione. Per mitigare questo collo di bottiglia, il kernel computazionale del risolutore SOR (`PoissonSolver_SOR`) è stato riscritto in linguaggio C++ e interfacciato con MATLAB tramite la funzionalità **MEX (MATLAB Executable)**. I vantaggi di questa soluzione ibrida sono:

1. **Codice Compilato:** Il C++ viene tradotto direttamente in linguaggio macchina, eliminando l'overhead dell'interprete.
2. **Gestione della Memoria:** L'uso di puntatori e l'accesso diretto alla memoria contigua (rispettando l'ordinamento *column-major* di MATLAB) permettono di sfruttare al meglio la cache della CPU.
3. **Aggiornamento In-Place:** La matrice ψ viene aggiornata direttamente in memoria senza copie intermedie.

L'utilizzo della routine MEX ha portato a una riduzione del tempo di calcolo del risolutore di oltre un ordine di grandezza rispetto alla controparte scritta in script MATLAB puro.

4.3 Integrazione Temporale Multi-Step (Adams-Bashforth)

Come alternativa allo schema *single-step* Runge-Kutta 4 (RK4), è stata valutata l'implementazione di metodi *multi-step* espliciti, come **Adams-Bashforth** (es. AB2 o AB3). Mentre RK4 richiede 4 valutazioni della funzione residuo $\mathcal{F}(\zeta)$ per ogni passo temporale (risultando costoso se $\mathcal{F}$ è complessa), un metodo Adams-Bashforth utilizza la "storia" temporale della soluzione per avanzare:

$$\zeta^{n+1} = \zeta^n + \Delta t \sum_{j=0}^k b_j \mathcal{F}(\zeta^{n-j}) \quad (39)$$

Ad esempio, lo schema AB2 richiede una sola valutazione di funzione per step, riutilizzando quella calcolata al passo precedente. Questo approccio riduce il

costo computazionale per singolo passo temporale, al prezzo di un dominio di stabilità generalmente più ridotto e della necessità di una procedura di avvio (start-up) mediante un metodo single-step (come RK4) per i primi istanti.

5 Analisi dei Risultati: Evoluzione della Scia e Conservazione

In questa sezione viene presentata l'evoluzione temporale del campo di moto attraverso la visualizzazione delle *streaklines*. Le particelle traccianti vengono rilasciate continuamente da un set di iniettori posti a monte dell'ostacolo. La colorazione delle linee non è puramente estetica ma rappresenta l'intensità e il segno della vorticità locale ζ :

- **Rosso:** Vorticità positiva ($\zeta > 0$, rotazione antioraria);
- **Blu:** Vorticità negativa ($\zeta < 0$, rotazione oraria);
- **Nero/Bianco:** Flusso irrotazionale ($\zeta \approx 0$).

Le frecce nere sovrapposte rappresentano il campo vettoriale di velocità istantanea.

5.1 Fase Iniziale e Transitorio ($t = 0.3 - t = 1.0$)

Nei primissimi istanti della simulazione, il flusso mantiene una simmetria rispetto all'asse orizzontale passante per il centro del cilindro.

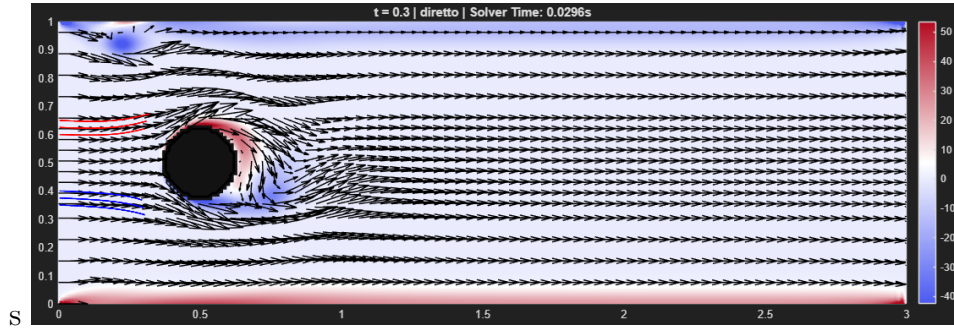


Figure 1: Istante $t = 0.3s$: Formazione della zona di ricircolo simmetrica.

Come osservabile in Figura 1 ($t = 0.3s$), le linee di corrente aderiscono alla parte anteriore del cilindro e si separano nella parte posteriore, formando due bolle di ricircolo controrotanti (i vortici gemelli) che rimangono inizialmente attaccate al corpo. In questa fase, le *streaklines* sono ancora regolari e non mostrano oscillazioni, indicando un regime quasi-stazionario instabile.

Successivamente (Figura 2, $t = 1s$), le perturbazioni numeriche (o fisiche, introdotte deliberatamente) iniziano ad amplificarsi. Le bolle di ricircolo si allungano e la simmetria speculare inizia a rompersi. Le *streaklines* iniziano a ondulare nella zona di scia lontana, preludio dell'instabilità di von Kármán.

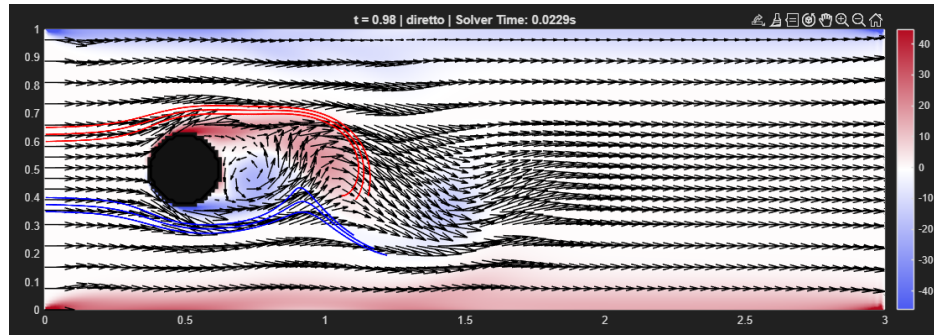


Figure 2: Istante $t = 1s$: Rottura della simmetria e inizio dell'instabilità.

5.2 Regime Completamente Sviluppato ($t = 5.0s$)

A tempi lunghi, il flusso raggiunge un regime periodico stabile caratterizzato dal distacco alternato di vortici (vortex shedding).

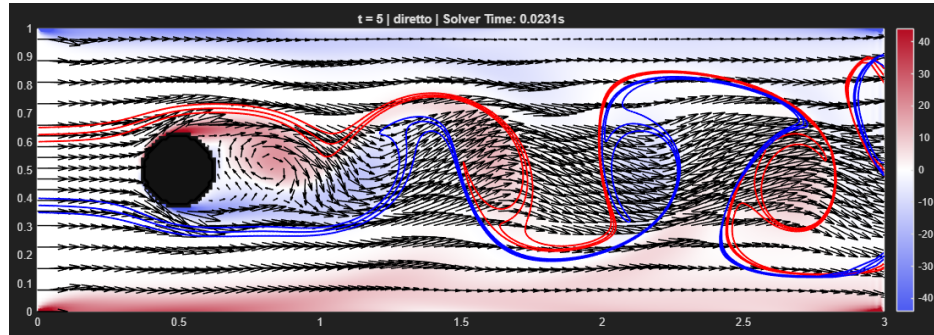


Figure 3: Istante $t = 5.0$: Scia di von Kármán completamente sviluppata.

La Figura 3 mostra chiaramente la **Scia di von Kármán**. Si nota l'alternanza di strutture vorticosi coerenti: un vortice orario (blu) si stacca dalla parte superiore, seguito da un vortice antiorario (rosso) dalla parte inferiore. Le *streaklines* non sono più linee tese ma si avvolgono a spirale all'interno dei nuclei vorticosi, evidenziando il trasporto di massa e la forte

miscelazione nella scia. Questo comportamento conferma la capacità del modello numerico di catturare le non-linearità convettive.

5.3 Analisi dell'Enstrofia e Singolarità Iniziale

Per valutare la robustezza e la conservatività dello schema numerico, è stata monitorata l'evoluzione dell'**Enstrofia Globale** $\Omega(t)$, definita (in 2D) come l'integrale del quadrato della vorticità sul dominio:

$$\Omega(t) = \frac{1}{2} \int_D \zeta^2 dA \approx \frac{h^2}{2} \sum_{i,j} \zeta_{i,j}^2 \quad (40)$$

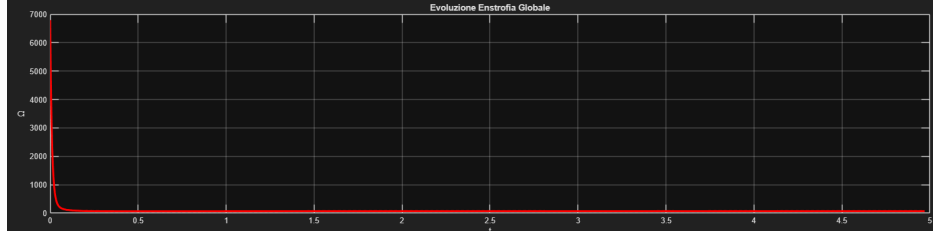


Figure 4: Evoluzione temporale dell'enstrofia globale.

Dal grafico in Figura 4 si osserva un comportamento caratteristico:

1. **Singolarità all'avvio** ($t \rightarrow 0$): Si registra un picco estremamente elevato di enstrofia nei primissimi istanti. Questo fenomeno è fisico e numerico, dovuto all'*Impulsive Start*. All'istante $t = 0$, il fluido è fermo (o ha velocità uniforme) mentre le pareti impongono istantaneamente la condizione di aderenza ($u = 0$). Matematicamente, questo crea una discontinuità di velocità sulla superficie del corpo, che corrisponde a uno strato limite di spessore nullo e quindi a un gradiente di velocità (e una vorticità) tendente all'infinito ($\zeta \propto \partial u / \partial n \rightarrow \infty$).
2. **Decadimento Viscoso**: Immediatamente dopo l'avvio, il termine diffusivo dell'equazione di Navier-Stokes ($\nu \nabla^2 \zeta$) agisce "spalmando" la vorticità dalla parete verso l'interno del dominio. Questo regolarizza la soluzione, ispessisce lo strato limite e fa crollare il valore dell'enstrofia verso valori finiti.

3. **Regime Asintotico:** Per $t > 1$, l'enstrofia si stabilizza su un valore medio quasi costante, indicando un equilibrio tra la produzione di vorticità alle pareti e la dissipazione viscosa nel dominio.

6 Implementazione e Analisi dei Casi Studio

In questo capitolo vengono analizzati e confrontati diversi approcci numerici per la risoluzione del sistema ψ - ζ . L'analisi si articola in due sezioni principali: la prima valuta l'efficienza dei solutori per l'equazione ellittica di Poisson (metodi diretti vs iterativi), mentre la seconda esamina l'impatto dei diversi schemi di integrazione temporale sulla stabilità e accuratezza della soluzione.

Tutti i casi studio condividono:

- la medesima discretizzazione spaziale su griglia collocata;
- lo stesso trattamento del termine convettivo (forma skew-symmetric);
- identiche condizioni al contorno e parametri fisici.

6.1 Setup della Simulazione

Tutte le simulazioni, salvo diversa indicazione, fanno riferimento al flusso intorno ad un cilindro circolare in un canale rettangolare. La geometria e la griglia sono definite dai parametri riportati in Tabella 1.

Parametro	Simbolo	Valore
Dominio (Altezza)	L_y	1.0
Dominio (Larghezza)	L_x	3.0 ($AR = 3$)
Raggio Cilindro	R	$L_y/8$
Posizione Centro	(x_c, y_c)	$(L_x/6, L_y/2)$
Nodi in y	N_y	80
Nodi in x	N_x	240
Passo spaziale	h	$1/80 \approx 0.0125$
Numero di Reynolds	Re	500
Tempo finale	T	30
CFL Convettivo	Cou	0.3
CFL Diffusivo	β	0.5

Table 1: Parametri geometrici, fisici e numerici della simulazione di riferimento.

Il passo temporale Δt è calcolato dinamicamente all’inizio della simulazione per garantire la stabilità secondo il criterio più restrittivo:

$$\Delta t \leq \min \left(\frac{Cou * th}{u_{max}}, \frac{\beta * Re * h^2}{4} \right) \quad (41)$$

6.2 Analisi dei Solutori per l’Equazione di Poisson

La risoluzione dell’equazione ellittica $\nabla^2 \psi = \zeta$ rappresenta il costo computazionale dominante in ogni passo temporale. Sono stati confrontati due approcci radicalmente differenti: i metodi diretti e i metodi iterativi.

Al fine di operare un confronto oggettivo tra le diverse strategie risolutive implementate, è stato definito un indicatore legato all’efficienza computazionale

6.2.1 Efficienza: Tempo Medio di Calcolo per Step ($\bar{T}_{step}$)

Poiché la risoluzione dell’equazione di Poisson ($\nabla^2 \psi = \zeta$) rappresenta l’operazione più onerosa dell’intero algoritmo (il ”collo di bottiglia”), il codice è stato strumentato per misurare il tempo di esecuzione di questa specifica fase ad ogni

iterazione temporale. Utilizzando le funzioni di timing del sistema (`tic` e `toc`), viene registrato l'intervallo di tempo T_i necessario al solutore per completare il calcolo al passo i -esimo. Il parametro di confronto utilizzato è il **Tempo Medio per Step**, calcolato come media aritmetica sui N_t passi totali della simulazione:

$$\bar{T}_{step} = \frac{1}{N_t} \sum_{i=1}^{N_t} T_i \quad (42)$$

- Per il **Metodo Diretto**, questo tempo include la risoluzione del sistema lineare sparso (back-substitution). Essendo la matrice fattorizzata una sola volta (o ri-analizzata efficientemente), questo tempo tende ad essere costante.
- Per i **Metodi Iterativi (SOR)**, questo tempo rappresenta la durata del ciclo `while` fino al raggiungimento della convergenza (residuo $< 10^{-5}$). Questo valore è sensibile all'implementazione software (Script vs C++/MEX).

Il confronto prestazionale (Tabella 2) evidenzia come l'implementazione compilata renda il metodo iterativo competitivo.

Tipologia Solutore	Tempo Medio per Step [s]
Diretto	0.02949
SOR	0.08327
SOR (C++ MEX)	0.00954

Table 2: Confronto indicativo dei tempi di calcolo per la risoluzione di Poisson.

6.3 Analisi Comparativa delle Strategie di Integrazione Temporale

In questa sezione si valuta l'impatto dello schema di integrazione temporale sulla stabilità e sulla qualità della soluzione fisica, mantenendo fissato il solutore spaziale. Sono stati confrontati tre schemi, le cui leggi di aggiornamento (indicando con $\mathcal{F}(\zeta)$ il residuo spaziale) sono definite come segue:

1. **Runge-Kutta 4 (RK4):** Metodo *single-step* esplicito. La soluzione al passo $n + 1$ è ottenuta come media pesata di quattro stadi intermedi ($k_1 \dots k_4$):

$$\zeta^{n+1} = \zeta^n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4) \quad (43)$$

È un metodo robusto ma costoso, richiedendo **4 valutazioni** funzionali per step.

2. **Adams-Bashforth 2 (AB2):** Metodo *multi-step* lineare del secondo ordine. Utilizza la "storia" della soluzione ai passi n e $n - 1$:

$$\zeta^{n+1} = \zeta^n + \frac{\Delta t}{2} (3\mathcal{F}(\zeta^n) - \mathcal{F}(\zeta^{n-1})) \quad (44)$$

3. **Adams-Bashforth 3 (AB3):** Metodo *multi-step* del terzo ordine, che estende lo storico fino a $n - 2$ per aumentare l'accuratezza formale:

$$\zeta^{n+1} = \zeta^n + \frac{\Delta t}{12} (23\mathcal{F}(\zeta^n) - 16\mathcal{F}(\zeta^{n-1}) + 5\mathcal{F}(\zeta^{n-2})) \quad (45)$$

6.3.1 Confronto di Stabilità e Robustezza

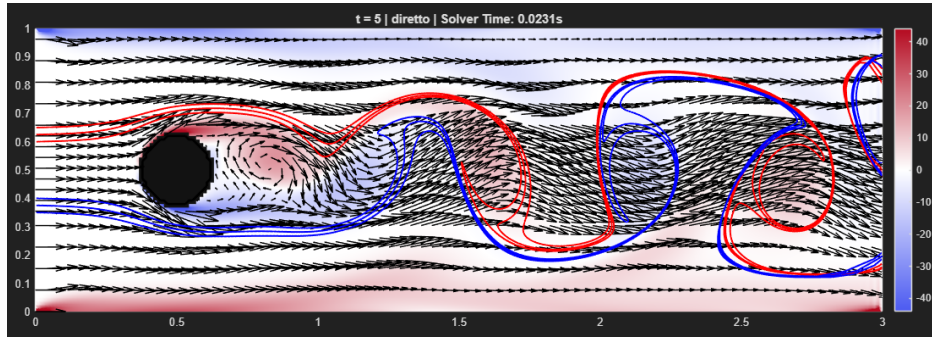
Le simulazioni condotte con i parametri di riferimento ($Re = 500$ e $Cou = 0.2$) hanno mostrato che tutti e tre i metodi convergono a una soluzione fisicamente coerente. In questo regime conservativo, le *streaklines* prodotte da AB2 e AB3 sono pressoché indistinguibili da quelle del RK4 (Figura 5), confermando che, se il passo temporale è sufficientemente piccolo, l'errore di troncamento dei metodi multi-step è accettabile.

Tuttavia, le differenze emergono drasticamente analizzando i margini di stabilità. È stata effettuata un'analisi di sensibilità incrementando il numero di Courant da 0.3 a 0.4 (il che corrisponde ad un aumento del passo temporale Δt , rendendo la condizione di stabilità più stringente).

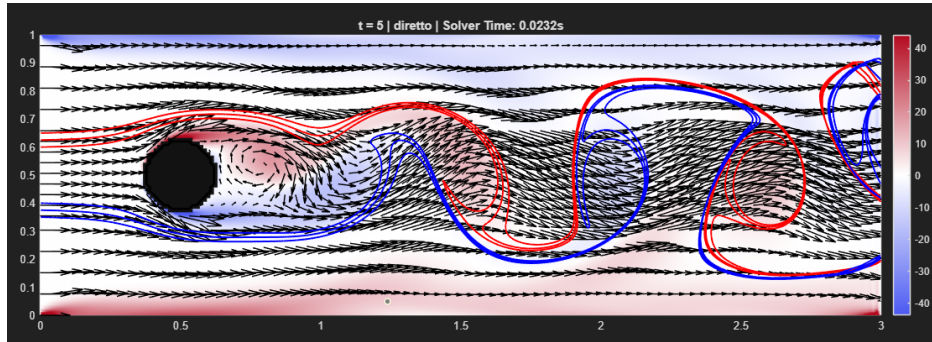
- **Instabilità dei metodi AB:** Con $Cou = 0.3$, lo schema Adams-Bashforth 3 (AB3) fallisce. La soluzione diverge rapidamente dopo pochi istanti temporali, portando l'enstrofia globale a valori infiniti. Questo comportamento è coerente con la teoria: i metodi multi-step espliciti possiedono regioni di stabilità ridotte, rendendoli vulnerabili in problemi dominati dalla convezione. Con $Cou = 0.4$, invece, entrambi i metodi (AB2, AB3) falliscono.

- **Robustezza del RK4:** Nelle stesse condizioni ($Cou = 0.4$), lo schema RK4 rimane stabile. Sebbene si osservi un incremento del picco iniziale di vorticità (dovuto alla minor risoluzione temporale della singolarità all'avvio), la simulazione supera il transitorio e si assesta correttamente.

Questa analisi conferma che, nonostante il costo computazionale per singolo step del RK4 sia quadruplo rispetto agli schemi AB, esso permette di utilizzare passi temporali più ampi senza rischi di divergenza, risultando complessivamente la scelta più affidabile per questo tipo di flussi.



(a) Soluzione RK4



(b) Soluzione Adams-Bashforth 3

Figure 5: Visualizzazione delle streaklines nel regime stabile ($Cou = 0.2$). Confronto verticale tra la soluzione RK4 (a) e quella ottenuta con AB3 (b).

6.4 Confronto Complessivo e Conclusioni

Dall'analisi comparativa emerge un quadro chiaro sulle strategie risolutive ottimali per questo tipo di problema:

1. **Solutore di Poisson:** L'utilizzo di un metodo iterativo SOR implementato in linguaggio compilato (C++/MEX) rappresenta la soluzione più scalabile, combinando bassa occupazione di memoria e tempi di calcolo confrontabili con i metodi diretti.
2. **Integrazione Temporale: RK4** si conferma la scelta più robusta. Sebbene computazionalmente più oneroso per singolo passo rispetto ai metodi Adams-Bashforth, la sua ampia regione di stabilità permette passi temporali più lunghi e garantisce una soluzione fisica priva di oscillazioni spurie.

7 Presentation of the Code

```

1 % Codice per la simulazione delle equazioni di Navier-Stokes 2D
  nella
2 % formulazione vorticit -funzione di corrente (Psi-Zita).
3 % Le equazioni sono integrate in una regione non connessa interna
  ad un
4 % rettangolo con un 'ostacolo' nel dominio.
5 %
6 % -----
7 % ->                                     ->
8 % ->                                     ->
9 % ->          -----                 ->
10 % ->          -       -                 ->
11 % ->          -       -                 ->
12 % ->          -----                 ->
13 % ->                                     ->
14 % ->                                     ->
15 % -----
16 %
17 %
18 % La collocazione delle variabili sul mesh   di tipo 'collocated',
  ovvero
19 % tutte le variabili sono collocate nei nodi di griglia, che cadono
  anche
20 % direttamente sul bordo.
21 %
22 % La indicizzazione delle variabili segue una notazione standard
  nella
23 % quale la variabile x   associata all'indice i, mentre la
  variabile y  
24 % associata all'indice j
25 %
26 %
27 %          Psi(i,j+1)
28 %              o
29 %              |
30 %              |
31 %              |
32 %          Psi(i-1,j) o-----o-----o Psi(i+1,j)

```



```

33 %                               Psi(i,j)
34 %                               |
35 %                               |
36 %                               o
37 %                               Psi(i,j-1)
38 close all; clear; clc;
39
40 global V U h hq Re G Nx Ny conv om tol
41 % -----
42
43 method = "diretto"; % Scegli il metodo per la discretizzazione
    spaziale: "diretto" (delsq) oppure "iterativo_sor/
    iterativo_sor_cpp" (SOR)
44 conv = 'skew'; % Discretizzazione spaziale Schema
    convettivo
45 time_method = "AB3"; % metodo temporale: scegli: "RK4", "AB2", "
    AB3"
46 om = 1.8; % Parametro di rilassamento per SOR (1 < om
    < 2)
47 tol = 1e-5; % Tolleranza per il metodo iterativo
48 % -----
49
50 % Domain geometry
51 Ly = 1;
52 AR = 3;
53 Lx = AR * Ly;
54 Nhy = 79;
55 Nhx = AR * Nhy;
56 Ny = Nhy + 1;
57 Nx = Nhx + 1;
58 y = linspace(0, Ly, Ny);
59 x = linspace(0, Lx, Nx);
60 h = x(2) - x(1);
61 hq = h*h;
62 [X, Y] = meshgrid(x, y);
63
64 % Boundary conditions su psi
65 Q = 1;
66 PsiN = Q * ones(1, Nx)';
67 PsiS = zeros(1, Nx)';

```

```

68 PsiW = linspace(0, Q, Ny)';
69 PsiE = PsiW;
70 PsiCyl = Q / 2;
71
72 % Physical parameters
73 T = 5;
74 Re = 500;
75 Cou = 0.2;
76 beta = 0.49;
77 uRef = Q / Ly;
78 Dt = min(Cou * h / uRef, beta * Re * hq);
79 Nt = ceil(T / Dt) + 1;
80 t = linspace(0, T, Nt);
81 Dtt = t(2) - t(1);
82
83 % Obstacle geometry (Cylinder)
84 Center = [Lx/6, Ly/2];
85 Radius = Ly / 8;
86
87 % Domain topology and discrete Laplacian operator
88 k = 0;
89 G = zeros(Nx, Ny);
90 for j = 2 : Ny-1
91     for i = 2 : Nx-1
92         if norm([x(i), y(j)] - Center) <= Radius %if (x(i)-center
(1))^2 + (y(j) - center(2))^2 < radius^2
93             G(i, j) = 0;
94         else
95             k = k + 1;
96             G(i, j) = k;
97         end
98     end
99 end
100
101 % PREPARAZIONE SOLUTORE
102 Lap = [];
103 if method == "diretto"
104     disp('Inizializzazione metodo DIRETTO (Matrice Laplaciana)...')
105     ;
106     Lap = -delsq(G) / hq;

```

```

106 else
107     disp('Inizializzazione metodo ITERATIVO (SOR)...');
108 end
109
110 % Set G=-1 inside obstacle for identification
111 for i = 2 : Nx-1
112     for j = 2 : Ny-1
113         if G(i, j) == 0
114             G(i, j) = -1;
115         end
116     end
117 end
118
119 % Domain and obstacle visualization
120 figurePosition = [0, 50, 1640, 1640/AR];
121 f = figure('Position', figurePosition, 'Units', 'pixels');
122 D = zeros(Nx, Ny);
123 D(G == -1) = 1;
124 D(G == 0) = 1;
125 D = D';
126 spy(D);
127 xlabel('\it i'); ylabel('\it j');
128 set(gca, 'YDir', 'normal', 'FontSize', 18);
129 drawnow;
130
131 % Array initialization
132 Ninc = k; %numero di incognite
133 zita = zeros(Ninc, 1);
134 PSI = zeros(Nx, Ny);
135 ZITA = PSI;
136 U = PSI;
137 V = PSI;
138
139 % Boundary conditions into the arrays
140 U(1, :) = uRef;          U(end, :) = uRef;    %velocita' costante
    parete 0-E
141 PSI(1, :) = PsiW;        PSI(end, :) = PsiE;
142 PSI(:, 1) = PsiS;        PSI(:, end) = PsiN;  %parete SUD psi=0,
    NORD psi=Q
143 PSI(G == -1) = PsiCyl;   %=Q/2

```

```

144
145 % Pathlines initial condition
146 xP{1} = [0; Ly/2 + 0.100]; xP{2} = [0; Ly/2 + 0.125]; xP{3} = [0;
    Ly/2 + 0.150];
147 xP{4} = [0; Ly/2 - 0.100]; xP{5} = [0; Ly/2 - 0.125]; xP{6} = [0;
    Ly/2 - 0.150];
148
149 % 6 Streaklines initialization
150 for s=1:6
151     xS{s} = zeros(2, Nt);
152     xS{s}(:, 1) = xP{s};
153     xS{s}(:, :) = repmat(xP{s}, 1, Nt);
154 end
155
156 % Graphics preparation
157 clf(f);
158 Uquiv = zeros(Nx, Ny); Vquiv = zeros(Nx, Ny);
159 iq = 1 : 3 : Nx;
160 jq = [1 : 6 : floor(Ny/3), floor(Ny/3+3) : 3 : floor(2*Ny/3), floor
    (2*Ny/3+6) : 6 : Ny];
161 Map = [linspace(75/255, 1, 500)', linspace(90/255, 1, 500)',
    linspace(241/255, 1, 500)']; ...
162     linspace(1, 180/255, 500)', linspace(1, 10/255, 500)',
    linspace(1, 32/255, 500)'];
163
164 % Variabili per analisi
165 enstrophy = NaN(Nt, 1);           %per valutare la conservativita'
    dell'estrofia
166 time_per_step = zeros(Nt, 1); %per valutare la velocita' del
    metodo
167
168
169 %%modifica per AB
170 RHS_hist = cell(3,1);
171 % RHS_hist{1} = f^{n-2}
172 % RHS_hist{2} = f^{n-1}
173 % RHS_hist{3} = f^{n}
174
175
176 % Time integration

```

```

177 disp('Avvio integrazione temporale...');
178 for it = 1 : Nt
179     time = t(it);
180
181     ZITA(G == -1) = 0;
182
183     % Vorticit al bordo (Thom)
184     ZITA = ThomFormulae(ZITA, PSI, G, hq);    %cc sulla zita
185
186
187     % Integrazione Eq. Vorticit (RK4 - AB2 - AB3)
188
189     % === Calcolo RHS corrente (f^n) ===
190     RHSn = ZitaRHS(time, ZITA, Re, h, hq, U, V, G, Nx, Ny);
191
192     switch time_method
193
194         case "RK4"
195             ZITA = RK4(time, ZITA, @(t,y) ZitaRHS(t,y,Re,h,hq,U,V,G
,Nx,Ny), Dt);
196
197         case "AB2"
198             if it <= 2
199                 % Startup con RK4
200                 ZITA = RK4(time, ZITA, @(t,y) ZitaRHS(t,y,Re,h,hq,U
,V,G,Nx,Ny), Dt);
201
202                 % Riempio storico progressivamente
203                 RHS_hist{it} = RHSn;
204             else
205                 % AB2: usa f^n e f^{n-1}
206                 ZITA = AB2step(ZITA, RHSn, RHS_hist{2}, Dt);
207
208                 % Shift storico
209                 RHS_hist{1} = RHS_hist{2};
210                 RHS_hist{2} = RHSn;
211             end
212
213         case "AB3"
214             if it <= 3

```

```

215         % Startup con RK4
216         ZITA = RK4(time, ZITA, @(t,y) ZitaRHS(t,y,Re,h,hq,U
,V,G,Nx,Ny), Dt);
217
218         RHS_hist{it} = RHSn;
219     else
220         % AB3: usa f^n, f^{n-1}, f^{n-2}
221         ZITA = AB3step(ZITA, RHSn, RHS_hist{3}, RHS_hist
{2}, Dt);
222
223         % Shift storico
224         RHS_hist{1} = RHS_hist{2};
225         RHS_hist{2} = RHS_hist{3};
226         RHS_hist{3} = RHSn;
227     end
228
229     otherwise
230         error("Metodo temporale non riconosciuto");
231 end
232
233
234 % Risoluzione Ellittica per PSI (Poisson)
235 tic; % Start timer risolutore
236
237 if method == "diretto"
238     % Metodo Diretto (si usa delsq classico)
239     [PSI, zita] = zita2psi(zita, ZITA, PSI, Lap, G, Nx, Ny, hq,
PsiS, PsiN, PsiW, PsiE, PsiCyl);
240 elseif method == "iterativo_sor"
241     % Metodo Iterativo (SOR)
242     % Nota: Non restituisce 'zita' vettore, ma aggiorna
direttamente PSI matrice
243     PSI = PoissonSolver_SOR(ZITA, PSI, G, PsiS, PsiN, PsiW,
PsiE, PsiCyl);
244 else
245     PSI = PoissonSolver_SOR_CPP(ZITA, PSI, G, PsiS, PsiN, PsiW,
PsiE, PsiCyl, hq, om, tol);
246 end
247
248 time_per_step(it) = toc; % Stop timer

```

```

249
250 % Calcolo Enstrofia Globale (uguale per entrambi i casi)
251 % Integrale di zita^2 sul dominio fluido
252 enstrophy(it) = 0.5 * sum(ZITA(G > 0).^2) * hq;
253
254 % Calcolo Velocit
255 [U, V] = psi2uv(PSI, U, V, G, h, Nx, Ny, uRef);
256
257 % Aggiornamento Streaklines
258 for k = 1 : it
259     for s=1:6
260         xS{s}(:, k) = RK4(time, xS{s}(:, k), @(t, y_)
PathlinesRHS(t, y_, x, y, U, V, h, h, Nx, Ny), Dt);
261     end
262 end
263
264
265 % Plotting ogni 10 step
266 if mod(it, 10) == 0
267     ZITAg = ZITA;
268     ZITAg(G == -1) = NaN;
269
270     subplot(2,1,1); % Plot fluido sopra
271     pcolor(x, y, ZITAg');
272     colormap(Map);
273     shading interp;
274
275     vals = ZITA(G > 0); vals = vals(~isnan(vals));
276     if ~isempty(vals)
277         c_min = 0.7 * min(vals); c_max = 0.7 * max(vals);
278         if c_min < c_max; clim([c_min, c_max]); end
279     end
280     colorbar; hold on;
281
282     Uquiv(iq, jq) = U(iq, jq);
283     Vquiv(iq, jq) = V(iq, jq);
284     quiver(X(jq, iq), Y(jq, iq), Uquiv(iq, jq)', Vquiv(iq, jq)
', 3, 'Color', 'k');
285
286 % Disegno ostacolo generico

```

```

287         contour(X, Y, G', [0 0], 'k', 'LineWidth', 2);
288
289         % Plot Streaklines
290         indices_plot = linspace(1, it, min(it*10, 2000));
291         for s = 1:6
292             if it > 1
293                 % Filtro NaN per evitare crash interpolazione
294                 Xpts = xS{s}(1, 1:it); Ypts = xS{s}(2, 1:it);
295                 if any(isnan(Xpts)) || any(isnan(Ypts)); continue;
296             end
297
298             x_plot = interp1(1:it, Xpts, indices_plot, 'pchip')
299             ;
300             y_plot = interp1(1:it, Ypts, indices_plot, 'pchip')
301             ;
302             col = 'r'; if s > 3; col = 'b'; end
303             plot(x_plot, smooth(y_plot), 'LineWidth', 1.5, '
304             Color', col);
305         end
306     end
307
308     title(['t = ', num2str(time, 2), ' | ', char(method), ' |
309     Solver Time: ', num2str(time_per_step(it), '%.4f'), 's']);
310     axis image; axis([0, Lx, 0, Ly]);
311     hold off;
312
313     subplot(2,1,2); % Plot enstrofia sotto
314     plot(t(1:it), enstrophy(1:it), 'r', 'LineWidth', 2.5);
315     title('Evoluzione Enstrofia Globale');
316     xlabel('t'); ylabel('\Omega'); grid on; xlim([0, T]);
317
318     drawnow limitrate;
319 end
320
321 avg_time = mean(time_per_step);
322 fprintf('Simulazione conclusa. Metodo: %s. Tempo medio solver: %.5f
323         s\n', method, avg_time);

```



```

1 function yNew = RK4(t, yOld, RHSfunction, Dt)
2     t1 = t;
3     y1 = yOld;
4     f1 = RHSfunction(t1, y1);
5
6     t2 = t + Dt / 2;
7     y2 = yOld + Dt * 0.5 * f1;
8     f2 = RHSfunction(t2, y2);
9
10    t3 = t + Dt / 2;
11    y3 = yOld + Dt * 0.5 * f2;
12    f3 = RHSfunction(t3, y3);
13
14    t4 = t + Dt;
15    y4 = yOld + Dt * f3;
16    f4 = RHSfunction(t4, y4);
17
18    yNew = yOld + Dt * ((f1 + f4)./6 + (f2 + f3)./3);
19 end

```

```

1 function dxdt = PathlinesRHS(t, xP, x, y, U, V, Dx, Dy, Nx, Ny)
2
3     %localizzazione deglla cella in cui si trova la particella
4     i = floor(xP(1) / Dx) + 1;
5     j = floor(xP(2) / Dy) + 1;
6
7     % limiti indici
8     i = max(1, min(i, Nx-1));
9     j = max(1, min(j, Ny-1));
10
11     %coordinate del nodo in basso a sinistra deella cella in cui si
12     trova la particella
13     xNode = [x(i); y(j)];
14
15     %adimensionalizzazione della posizione della particella q nella
16     cella
17     csix = (xP(1) - xNode(1)) / Dx;
18     csiy = (xP(2) - xNode(2)) / Dy;
19
20     if i+1 <= Nx && j+1 <= Ny && i > 0 && j > 0
21         % Interpolazione bilineare
22         u = [1-csix, csix] * U([i, i+1], [j, j+1]) * [1-csiy; csiy
23         ];
24         v = [1-csix, csix] * V([i, i+1], [j, j+1]) * [1-csiy; csiy
25         ];
26     else
27         u = 0.05;
28         v = 0;
29     end
30
31     dxdt = [u; v];
32 end

```

```

1 function F = ZitaRHS(t, ZITA, Re, h, hq, U, V, G, Nx, Ny)
2 % RHS della equazione per Zita per il codice TunnelPsiZita
3 global conv
4 F = zeros(Nx,Ny);
5 for i = 2:Nx-1
6     for j = 2:Ny-1
7 % Convective term
8         if G(i,j)>0
9             switch conv
10                 case 'div'
11                     F(i,j) = -((U(i+1,j)*ZITA(i+1,j)-U(i-1,j)*ZITA(
12 i-1,j)))/(2*h)+...
13                                     (V(i,j+1)*ZITA(i,j+1)-V(i,j-1)*ZITA(
14 i,j-1))/(2*h) );
15                 case 'adv'
16                     F(i,j) = -( U(i,j)*(ZITA(i+1,j)-ZITA(i-1,j))
17 /(2*h)+...
18                                     V(i,j)*(ZITA(i,j+1)-ZITA(i,j-1))
19 /(2*h) );
20                 case 'skew'
21                     F(i,j) = -0.5*((U(i+1,j)*ZITA(i+1,j)-U(i-1,j)*
22 ZITA(i-1,j))/(2*h)+...
23                                     (V(i,j+1)*ZITA(i,j+1)-V(i,j-1)*
24 ZITA(i,j-1))/(2*h) );
25                     F(i,j) = F(i,j)-0.5*(U(i,j)*(ZITA(i+1,j)-ZITA(i
26 -1,j))/(2*h)+...
27                                     V(i,j)*(ZITA(i,j+1)-ZITA(i
28 ,j-1))/(2*h) );
29             end
30 % Diffusive term
31 F(i,j) = F(i,j) + (1/Re)*(ZITA(i+1,j)+ZITA(i-1,j)+ZITA(
32 i,j+1)+...
33                                     ZITA(i,j-1) - 4*ZITA(i,j))/hq
34 ;
35     end
36 end
37 end

```

```

1
2 function [PSI, zita] = zita2psi(zita, ZITA, PSI, Lap, G, Nx, Ny, hq
   , PsiS, PsiN, PsiW, PsiE, PsiCyl)
3
4 % Inizializzazione: Mappatura della vorticit  (ZITA 2D) nel
   vettore termine noto (zita 1D)
5 for i = 2:Nx-1
6     for j = 2:Ny-1
7         if G(i,j) > 0
8             % Mettiamo il valore di vorticit  nel punto k
   corrispondente
9             zita(G(i,j)) = ZITA(i,j);
10        end
11    end
12 end
13
14 % Modifica del termine noto per le Condizioni al Contorno (BCs)
15 % Ciclo su tutti i nodi interni per controllare i vicini
16 for i = 2:Nx-1
17     for j = 2:Ny-1
18         if G(i,j) > 0      % Se siamo in un nodo fluido
19             k = G(i,j);    % Indice lineare nel sistema
   risolutivo
20
21             % --- Controllo Pareti Esterne (G == 0) ---
22
23             % Parete Sud (vicino j-1 e' bordo cioe' e' 0)
24             if G(i,j-1) == 0; zita(k) = zita(k) - PsiS(i)/hq;
25         end
26
27             % Parete Nord (vicino j+1 e' bordo 0)
28             if G(i,j+1) == 0; zita(k) = zita(k) - PsiN(i)/hq;
29         end
30
31             % Parete Ovest (vicino i-1 e' bordo 0)
32             if G(i-1,j) == 0; zita(k) = zita(k) - PsiW(j)/hq;
33         end
34
35             % Parete Est (vicino i+1 e' bordo 0)
36             if G(i+1,j) == 0; zita(k) = zita(k) - PsiE(j)/hq;
37         end
38     end
39 end

```

```

32         % Controllo Pareti Ostacolo (G == -1)
33         % Se un vicino e' ostacolo, sottraiamo il valore
costante PsiCyl
34
35         if G(i,j+1) == -1; zita(k) = zita(k) - PsiCyl/hq;
end
36         if G(i,j-1) == -1; zita(k) = zita(k) - PsiCyl/hq;
end
37         if G(i+1,j) == -1; zita(k) = zita(k) - PsiCyl/hq;
end
38         if G(i-1,j) == -1; zita(k) = zita(k) - PsiCyl/hq;
end
39     end
40     end
41 end
42
43 % Risoluzione dell'equazione di Poisson (Sistema Lineare) con
metodo diretto
44 % Lap e' la matrice del Laplaciano discreto, zita e' il termine
noto modificato
45 psi = Lap \ zita;
46
47 % Riversamento del risultato nel vettore 2D PSI
48 PSI(G > 0) = psi;
49 end

```

```

1
2 function PHI = PoissonSolver_SOR(Zita, PSI_old, G, PsiS, PsiN, PsiW
   , PsiE, PsiCyl)
3 % Risolutore iterativo SOR per l'equazione di Poisson  $\nabla^2 \psi =$ 
   zita
4 % Le BCs sono Dirichlet non omogenee (PHI = cost != 0).
5
6 global Nx Ny hq om tol
7
8 PHI = PSI_old;
9 iter = 0;
10 normres = 1;
11 max_iter = 2000; % Limite sicurezza per evitare loop infiniti
12
13 while normres > tol && iter < max_iter
14     iter = iter + 1;
15     max_res = 0; % Per calcolare la norma infinito del residuo
16
17     for i = 2:Nx-1
18         for j = 2:Ny-1
19             if G(i,j) > 0 % Calcola solo sui nodi fluidi
20
21                 % Si recuperano i valori dai vicini (Gestione BCs)
22                 , in particolare se il vicino e' un punto del bordo allora si
23                 prende il valore noto di PSI altrimenti
24                 % si prende il valore in quei punti di PSI.
25
26                 % EST (i+1)
27                 if G(i+1,j) > 0;      valE = PHI(i+1,j);    % Fluido
28                 elseif G(i+1,j) == 0; valE = PsiE(j);      % Bordo
29
30                 Est
31
32                 else;                  valE = PsiCyl;      %
33
34                 Ostacolo
35
36                 end
37
38                 % OVEST (i-1)
39                 if G(i-1,j) > 0;      valW = PHI(i-1,j);    % Fluido
40                 elseif G(i-1,j) == 0; valW = PsiW(j);      % Bordo
41
42                 Ovest

```

```

33         else;                                valW = PsiCyl;          %
34     Ostacolo
35
36         % NORD (j+1)
37         if G(i,j+1) > 0;          valN = PHI(i,j+1);    % Fluido
38         elseif G(i,j+1) == 0; valN = PsiN(i);          % Bordo
39     Nord
40         else;                                valN = PsiCyl;          %
41     Ostacolo
42         end
43
44         % SUD (j-1)
45         if G(i,j-1) > 0;          valS = PHI(i,j-1);    % Fluido
46         elseif G(i,j-1) == 0; valS = PsiS(i);          % Bordo
47     Sud
48         else;                                valS = PsiCyl;          %
49     Ostacolo
50         end
51
52         % Formula di Aggiornamento SOR
53         % Laplaciano discreto: (E + W + N + S - 4P)/h^2 =
54     Zita
55         % P_new = (1-om)*P_old + om * 0.25 * (E + W + N + S
56         - h^2*Zita)
57
58         RHS_Term = hq * Zita(i,j);
59         Phi_Star = 0.25 * (valE + valW + valN + valS -
60     RHS_Term);
61
62         % Calcolo residuo locale (solo ogni 10 iterazioni
63     per velocita')
64         if mod(iter, 10) == 0
65             res_loc = abs(Phi_Star - PHI(i,j));
66             if res_loc > max_res; max_res = res_loc; end
67         end
68
69         % Aggiornamento
70     PHI(i,j) = (1 - om) * PHI(i,j) + om * Phi_Star;

```

```
64         end
65     end
66 end
67
68     if mod(iter, 10) == 0
69         normres = max_res;
70     end
71 end
72 end
```



```

1 #include "mex.h"
2 #include <cmath>
3 #include <algorithm>
4
5 /*
6  * PoissonSolver_SOR_CPP.cpp
7  * Risolutore SOR in C++ per l'equazione di Poisson nel modello Psi
    -Zita.
8  * * INPUTS (dati da MATLAB):
9  * 0: Zita (Nx x Ny double)
10 * 1: PSI_old (Nx x Ny double) - Initial Guess
11 * 2: G (Nx x Ny double) - Topological Matrix
12 * 3: PsiS (Vector double) - BC Sud
13 * 4: PsiN (Vector double) - BC Nord
14 * 5: PsiW (Vector double) - BC Ovest
15 * 6: PsiE (Vector double) - BC Est
16 * 7: PsiCyl (Scalar double) - BC Ostacolo
17 * 8: hq (Scalar double) - h^2
18 * 9: om (Scalar double) - Omega relaxation
19 * 10: tol (Scalar double) - Tolerance
20 * * OUTPUT:
21 * 0: PHI (Nx x Ny double) - Updated stream function
22 */
23
24 // Macro per accedere agli array lineari come matrici 2D (Column-
    Major order di MATLAB)
25 // i = riga (x), j = colonna (y), M = numero di righe (Nx)
26 #define IDX(i, j, M) ((i) + (j)*(M))
27
28 void mexFunction(int nlhs, mxArray *plhs[], int nrhs, const mxArray
    *prhs[]) {
29
30     // CONTROLLO INPUT
31     if (nrhs != 11) {
32         mexErrMsgIdAndTxt("Fluid:SOR:InvalidInput", "Servono 11
            argomenti di input.");
33     }
34
35     // ESTRAZIONE DATI

```

```

36 // Puntatori agli array di input
37 double *Zita = mxGetPr(prhs[0]);
38 double *PSI_In = mxGetPr(prhs[1]);
39 double *G = mxGetPr(prhs[2]);
40 double *PsiS = mxGetPr(prhs[3]);
41 double *PsiN = mxGetPr(prhs[4]);
42 double *PsiW = mxGetPr(prhs[5]);
43 double *PsiE = mxGetPr(prhs[6]);
44
45 // Scalari
46 double PsiCyl = mxGetScalar(prhs[7]);
47 double hq = mxGetScalar(prhs[8]);
48 double om = mxGetScalar(prhs[9]);
49 double tol = mxGetScalar(prhs[10]);
50
51 // Dimensioni (Nx corrisponde alle righe M, Ny alle colonne N)
52 const mwSize Nx = mxGetM(prhs[0]);
53 const mwSize Ny = mxGetN(prhs[0]);
54
55 // PREPARAZIONE OUTPUT
56 // Creiamo la matrice di uscita duplicando la PSI_old (initial
57 // guess)
58 plhs[0] = mxDuplicateArray(prhs[1]);
59 double *PHI = mxGetPr(plhs[0]);
60
61 // ALGORITMO SOR
62 int max_iter = 5000;
63 int iter = 0;
64 double normres = 1.0;
65
66 // Variabili temporanee per i valori dei vicini
67 double valN, valS, valE, valW;
68
69 while (normres > tol && iter < max_iter) {
70     iter++;
71     double max_res_loc = 0.0;
72
73     // Cicli sui nodi interni (1-based in MATLAB diventa 1..Nx
74     // -2 in C++)
75     // IMPORTANTE: Loop esterno su J (colonne) e interno su I (

```

```

74         righe)
           // per rispettare la memoria column-major e ottimizzare la
           cache.
75         for (mwSize j = 1; j < Ny - 1; j++) {
76             for (mwSize i = 1; i < Nx - 1; i++) {
77
78                 // Indice lineare del nodo corrente
79                 mwSize k = IDX(i, j, Nx);
80
81                 // Se siamo nel fluido (G > 0)
82                 if (G[k] > 0) {
83
84                     // --- EST (i+1) ---
85                     mwSize kE = IDX(i + 1, j, Nx);
86                     if (G[kE] > 0)         valE = PHI[kE];
87                     else if (G[kE] == 0) valE = PsiE[j]; // PsiE(j)
           in Matlab
88                     else                     valE = PsiCyl;
89
90                     // --- OVEST (i-1) ---
91                     mwSize kW = IDX(i - 1, j, Nx);
92                     if (G[kW] > 0)         valW = PHI[kW];
93                     else if (G[kW] == 0) valW = PsiW[j]; // PsiW(j)
           in Matlab
94                     else                     valW = PsiCyl;
95
96                     // --- NORD (j+1) ---
97                     mwSize kN = IDX(i, j + 1, Nx);
98                     if (G[kN] > 0)         valN = PHI[kN];
99                     else if (G[kN] == 0) valN = PsiN[i]; // PsiN(i)
           in Matlab
100                     else                     valN = PsiCyl;
101
102                     // --- SUD (j-1) ---
103                     mwSize kS = IDX(i, j - 1, Nx);
104                     if (G[kS] > 0)         valS = PHI[kS];
105                     else if (G[kS] == 0) valS = PsiS[i]; // PsiS(i)
           in Matlab
106                     else                     valS = PsiCyl;
107

```

```

108         // --- CALCOLO SOR ---
109         double RHS_Term = hq * Zita[k];
110         double Phi_Star = 0.25 * (valE + valW + valN +
    valS - RHS_Term);
111
112         // Calcolo residuo (ogni 10 iterazioni per
    performance)
113         if (iter % 10 == 0) {
114             double res = std::abs(Phi_Star - PHI[k]);
115             if (res > max_res_loc) max_res_loc = res;
116         }
117
118         // Aggiornamento In-Place
119         PHI[k] = (1.0 - om) * PHI[k] + om * Phi_Star;
120     }
121 }
122
123
124     // Aggiorna la norma del residuo ogni 10 iterazioni
125     if (iter % 10 == 0) {
126         normres = max_res_loc;
127     }
128 }
129 }

```

```

1 function [U, V] = psi2uv(PSI, U, V, G, h, Nx, Ny, uRef)
2     i = 2:Nx-1;    j = 2:Ny-1;
3
4     U(i, j) = (PSI(i, j+1) - PSI(i, j-1)) / (2 * h);
5     V(i, j) = -(PSI(i+1, j) - PSI(i-1, j)) / (2 * h);
6
7     U(G < 0) = 0;
8     V(G < 0) = 0;
9
10    U(1, :) = uRef;
11    U(end, :) = uRef;
12 end

```

```

1 function yNew = AB2step(yN, fN, fNm1, Dt)
2 % Adams-Bashforth 2-step explicit
3 %  $y^{n+1} = y^n + Dt*(3/2 f^n - 1/2 f^{n-1})$ 
4
5 yNew = yN + Dt*(1.5*fN - 0.5*fNm1);
6
7 end

```

```

1 function yNew = AB3step(yN, fN, fNm1, fNm2, Dt)
2 % Adams-Bashforth 3-step explicit
3 %  $y^{n+1} = y^n + Dt*(23/12 f^n - 16/12 f^{n-1} + 5/12 f^{n-2})$ 
4
5 yNew = yN + Dt*( (23/12)*fN - (16/12)*fNm1 + (5/12)*fNm2 );
6
7 end

```

References

- [1] J. C. Tannehill, D. A. Anderson and R. H. Pletcher, *Computational Fluid Mechanics and Heat Transfer*, 2nd edition. CRC Press (1997).

List of Figures

1	Istante $t = 0.3s$: Formazione della zona di ricircolo simmetrica.	22
2	Istante $t = 1s$: Rottura della simmetria e inizio dell'instabilità.	23
3	Istante $t = 5.0$: Scia di von Kármán completamente sviluppata.	23
4	Evoluzione temporale dell'enstrofia globale.	24
5	Visualizzazione delle streaklines nel regime stabile ($Cou = 0.2$). Confronto verticale tra la soluzione RK4 (a) e quella ottenuta con AB3 (b).	29

List of Tables

1	Parametri geometrici, fisici e numerici della simulazione di riferimento.	26
2	Confronto indicativo dei tempi di calcolo per la risoluzione di Poisson.	27